

[Refactoring](#) [Agile](#) [Architecture](#) [About](#) [Thoughtworks](#)  

Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl



This article is part of “[Exploring Gen AI](#)”. A series capturing Thoughtworks technologists' explorations of using gen ai technology for software development.

15 October 2025

Birgitta Böckeler



Birgitta is a Distinguished Engineer and AI-assisted delivery expert at Thoughtworks. She has over 20 years of experience as a software developer, architect and technical leader.

I’ve been trying to understand one of the latest AI coding buzzword: Spec-driven development (SDD). I looked at three of the tools that label themselves as SDD tools and tried to untangle what it means, as of now.

Definition

Like with many emerging terms in this fast-paced space, the definition of “spec-driven development” (SDD) is still in flux. Here’s what I can gather from how I have seen it used so far: Spec-driven development means writing a “spec” before writing code with AI (“documentation first”). The spec becomes the source of truth for the human and the AI.

GitHub: “In this new world, *maintaining software means evolving specifications*. [...] The lingua franca of development moves to a higher level, and code is the last-mile approach.”

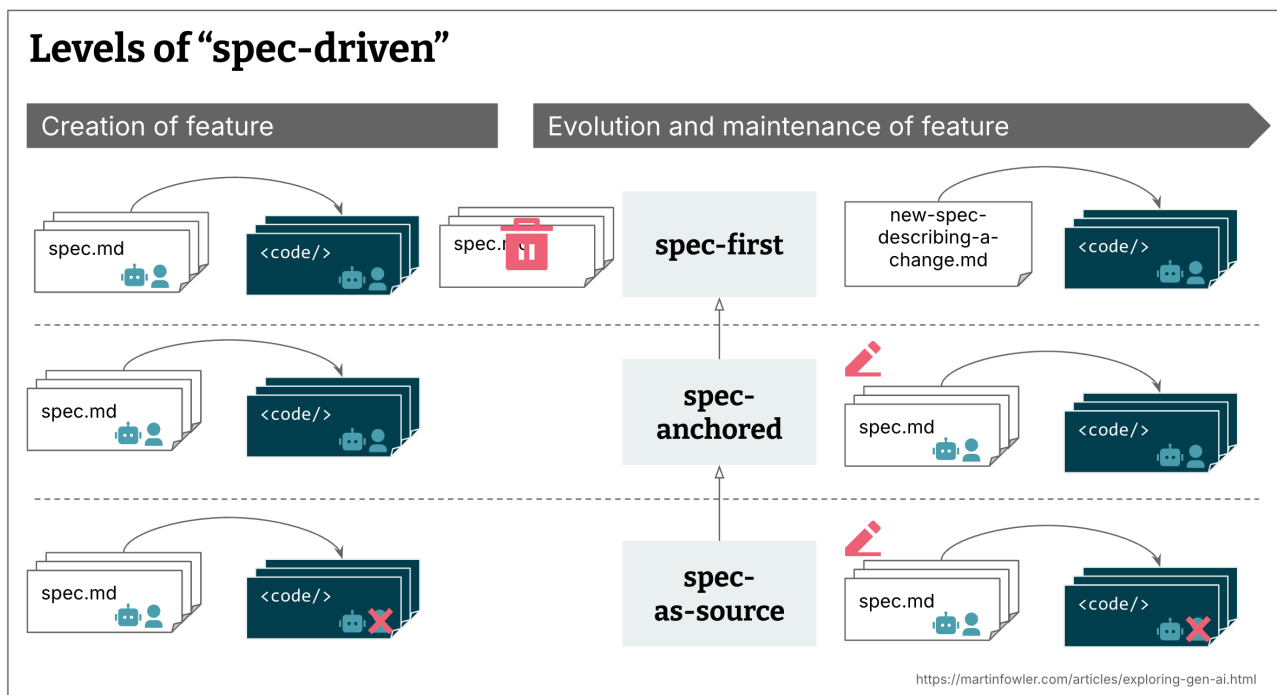
Tessl: “A development approach where *specs – not code – are the primary artifact*. Specs describe intent in structured, testable language, and agents generate code to match them.”

After looking over the usages of the term, and some of the tools that claim to be implementing SDD, it seems to me that in reality, there are multiple implementation

levels to it:

1. **Spec-first:** A well thought-out spec is written first, and then used in the AI-assisted development workflow for the task at hand.
2. **Spec-anchored:** The spec is kept even after the task is complete, to continue using it for evolution and maintenance of the respective feature.
3. **Spec-as-source:** The spec is the main source file over time, and only the spec is edited by the human, the human never touches the code.

All SDD approaches and definitions I've found are spec-first, but not all strive to be spec-anchored or spec-as-source. And often it's left vague or totally open what the spec maintenance strategy over time is meant to be.



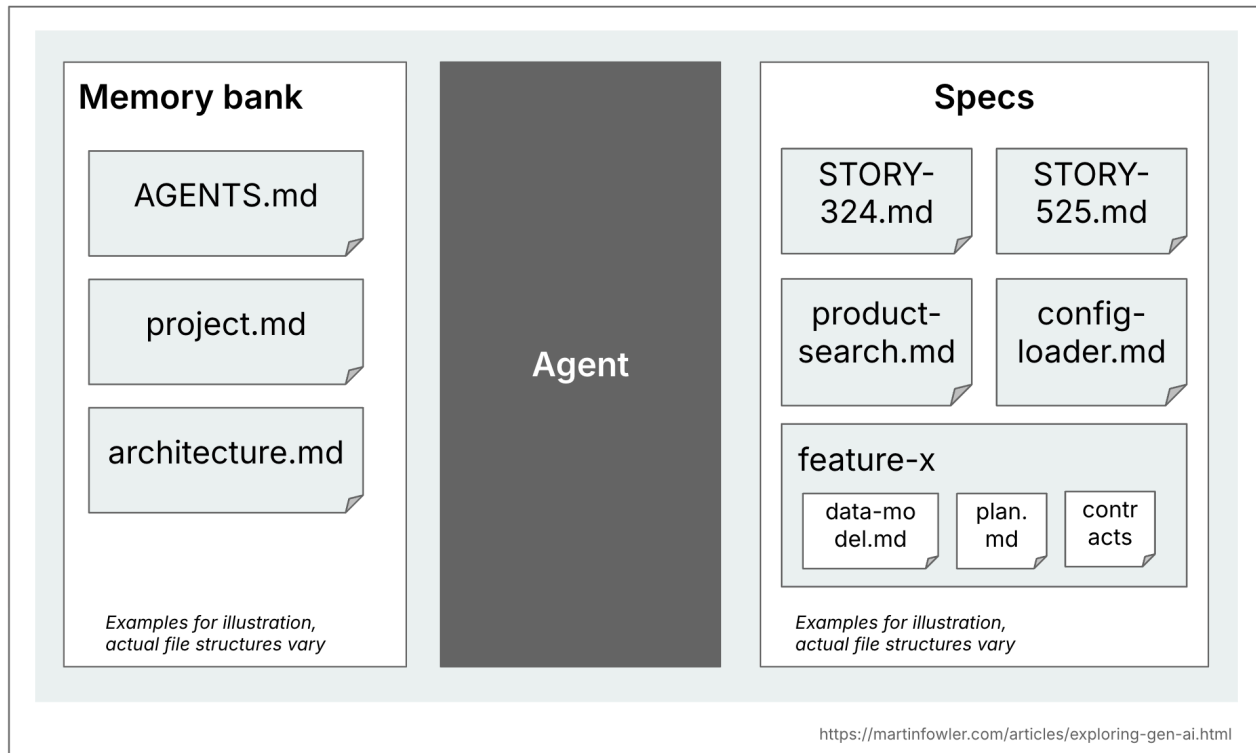
What is a spec?

The key question in terms of definitions of course is: What is a spec? There doesn't seem to be a general definition, the closest I've seen to a consistent definition is the comparison of a spec to a "Product Requirements Document".

The term is quite overloaded at the moment, here is my attempt at defining what a spec is:

A spec is a structured, behavior-oriented artifact - or a set of related artifacts - written in natural language that expresses software functionality and serves as guidance to AI coding agents. Each variant of spec-driven development defines their approach to a spec's structure, level of detail, and how these artifacts are organized within a project.

There is a useful difference to be made I think between specs and the more general context documents for a codebase. That general context are things like rules files, or high level descriptions of the product and the codebase. Some tools call this context a **memory bank**, so that's what I will use here. These files are relevant across all AI coding sessions in the codebase, whereas specs only relevant to the tasks that actually create or change that particular functionality.



The challenge with evaluating SDD tools

It turns out to be quite time-consuming to evaluate SDD tools and approaches in a way that gets close to real usage. You would have to try them out with different sizes of problems, greenfield, brownfield, and really take the time to review and revise the intermediate artifacts with more than just a cursory glance. Because as [GitHub's blog post about spec-kit](#) says: "Crucially, your role isn't just to steer. It's to verify. At each phase, you reflect and refine."

For two of the three tools I tried it also seems to be even more work to introduce them into an existing codebase, therefore making it even harder to evaluate their usefulness for brownfield codebases. Until I hear usage reports from people using them for a period of time on a "real" codebase, I still have a lot of open questions about how this works in real life.

That being said - let's get into three of these tools. I will share a description of how they work first (or rather how I think they work), and will keep my observations and

questions for the end. Note that these tools are very fast evolving, so they might have already changed since I used them in September.

Kiro

Kiro is the simplest (or most lightweight) one of the three I tried. It seems to be mostly spec-first, all the examples I have found use it for a task, or a user story, with no mention of how to use the requirements document in a spec-anchored way over time, across multiple tasks.

Workflow: Requirements → Design → Tasks

Each workflow step is represented by one markdown document, and Kiro guides you through those 3 workflow steps inside of its VS Code based distribution.

Requirements: Structured as a list of requirements, where each requirement represents a “User Story” (in “As a...” format) with acceptance criteria (in “GIVEN... WHEN... THEN...” format)

The screenshot shows the Kiro IDE interface with three tabs: requirements.md, design.md, and tasks.md. The active tab is requirements.md, which displays a document titled '# Requirements Document'. The document structure is as follows:

```

1  # Requirements Document
3  ## Introduction
   sophisticated, centralized transformation function that converts
   technical category names into proper human-readable labels with
   correct spacing and capitalization.
6
7  < ## Requirements
8
9  < ### Requirement 1
10
11  **User Story:** As a user viewing the dashboard filter labels, I
   want category names to be displayed in proper human-readable
   format, so that I can easily understand and interact with the
   filtering options.
12
13 < #### Acceptance Criteria
14
15  1. WHEN a category name contains hyphens THEN the system SHALL
   convert hyphens to spaces and capitalize each word
16  2. WHEN a category name is in camelCase THEN the system SHALL
   insert spaces before capital letters and capitalize each word
   appropriately
17  3. WHEN a category name contains mixed formats (kebab-case with
   camelCase) THEN the system SHALL handle both transformations
   correctly
18  4. WHEN the category "client-research" is processed THEN the system
   SHALL display "Client Research"
19  5. WHEN the category "deliveryManagement" is processed THEN the
   system SHALL display "Delivery Management"
20
21 > ### Requirement 2 ...
31
32 > ### Requirement 3 ...
41
42 > ### Requirement 4 ...

```

The interface includes a breadcrumb trail: .kiro > specs > category-label-enhancement > requirements.md > # Requirements Document. Below the breadcrumb, there are tabs for 'Spec: category-label-enhancement', '1 Requirements', '2 Design', and '3 Task list', along with a 'Refine' button. A line number indicator is visible on the left side of the code editor.

Design: In my attempt, the design document consisted of the sections seen in the screenshot below. I only have the results of one of my attempts still, so I'm not sure if this is a consistent structure, or if it changes depending on the task.

requirements.md U

design.md U ×

tasks.md U

.kiro > specs > category-label-enhancement > design.md > # Design Document > @

Spec: category-label-enhancement 1 Requirements 2 Design 3 Task list

1 # Design Document

10

11 ## Architecture

12

13 ### Component Architecture

14

15 ...

16

17

UI Components

Dashboard
Filter Tags

Sidebar
Navigation

Card Categories
Tags

18

19

20

21

22

23

24

25

categoryLabelUtils.js

toCategoryLabel(category)

Input: "client-research", "deliveryManagement"

Output: "Client Research", "Delivery Management"

26

27

28

29

30

31

32

33

34 ...

35

36 > ### Data Flow ...

41

42 ## Components and Interfaces

43

44 > ### Core Utility Function ...

58

59 > ### Integration Points ...

79

80 > ## Data Models ...

102

103 > ## Error Handling ...

115

116 > ## Testing Strategy ...

163

164 > ## Implementation Approach ...

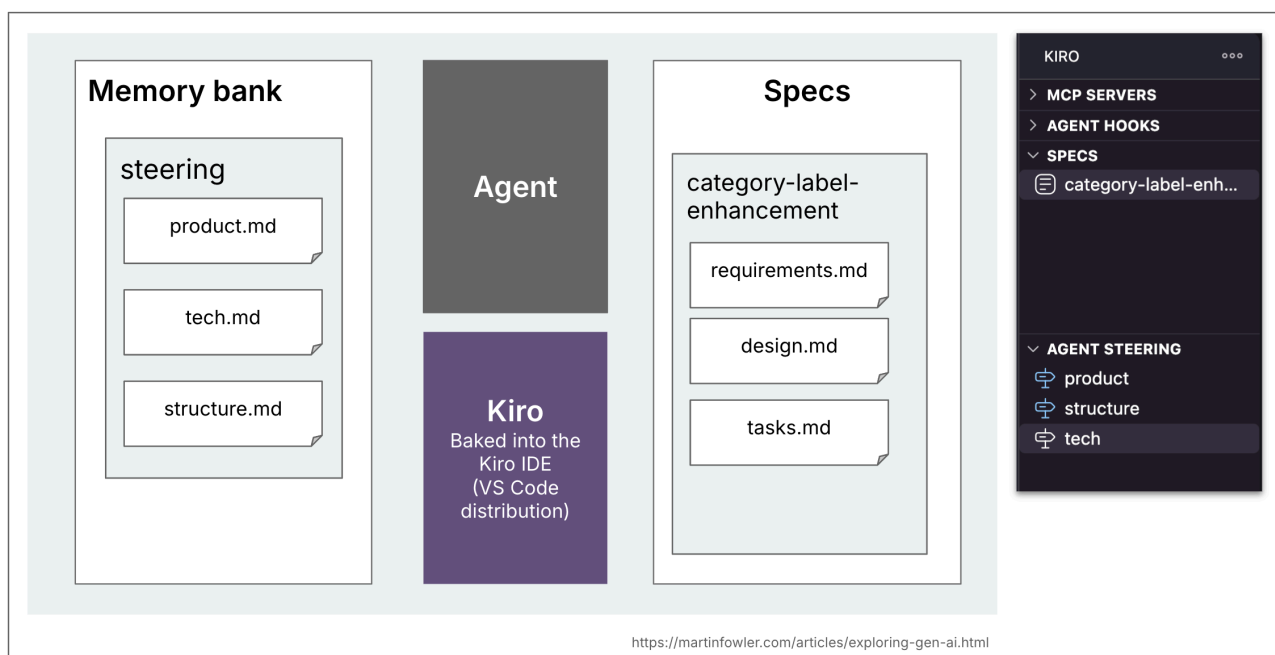
185

186 > ## Migration Strategy ...

Tasks: A list of tasks that trace back to the requirement numbers, and that get some extra UI elements to run tasks one by one, and review changes per task.

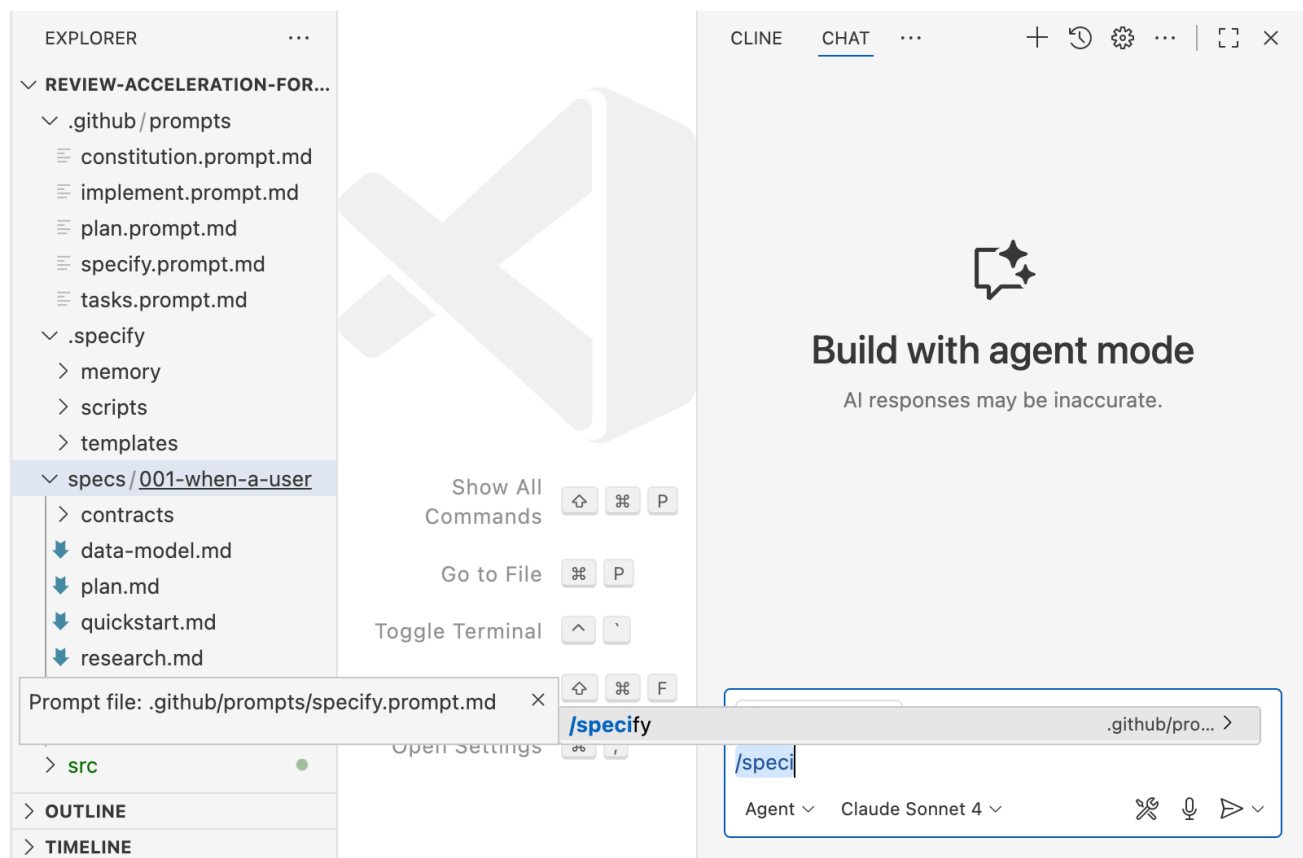
The screenshot shows the Kiro IDE interface with three tabs: requirements.md, design.md, and tasks.md. The 'tasks.md' tab is active, displaying a task list for 'category-label-enhancement'. The task list is organized into three sections: 'Requirements', 'Design', and 'Task list'. The 'Task list' section shows a task in progress, '1. Create core utility function and test suite', with sub-tasks for creating a utility function, implementing transformation logic, and handling edge cases. The task is linked to requirements 1.1, 1.2, 1.3, 4.1, 4.2, and 4.3. Below the task list, there is a 'Start task' button and a '2. Update dashboard filter tags implementation' task.

Kiro also has the concept of a memory bank, they call it “steering”. Its contents are flexible, and their workflow doesn’t seem to rely on any specific files being there (I made my usage attempts before I even discovered the steering section). The default topology created by Kiro when you ask it to generate steering documents is product.md, structure.md, tech.md.



Spec-kit

Spec-kit is GitHub's version of SDD. It is distributed as a CLI that can create workspace setups for a wide range of common coding assistants. Once that structure is set up, you interact with spec-kit via slash commands in your coding assistant. Because all of its artifacts are put right into your workspace, this is the most customizable one of the three tools discussed here.



Workflow: Constitution → ⚡ Specify → Plan → Tasks ⚡

Spec-kit's memory bank concept is a prerequisite for the spec-driven approach. They call it a **constitution**. The constitution is supposed to contain the high level principles that are “immutable” and should always be applied, to every change. It's basically a very powerful rules file that is heavily used by the workflow.

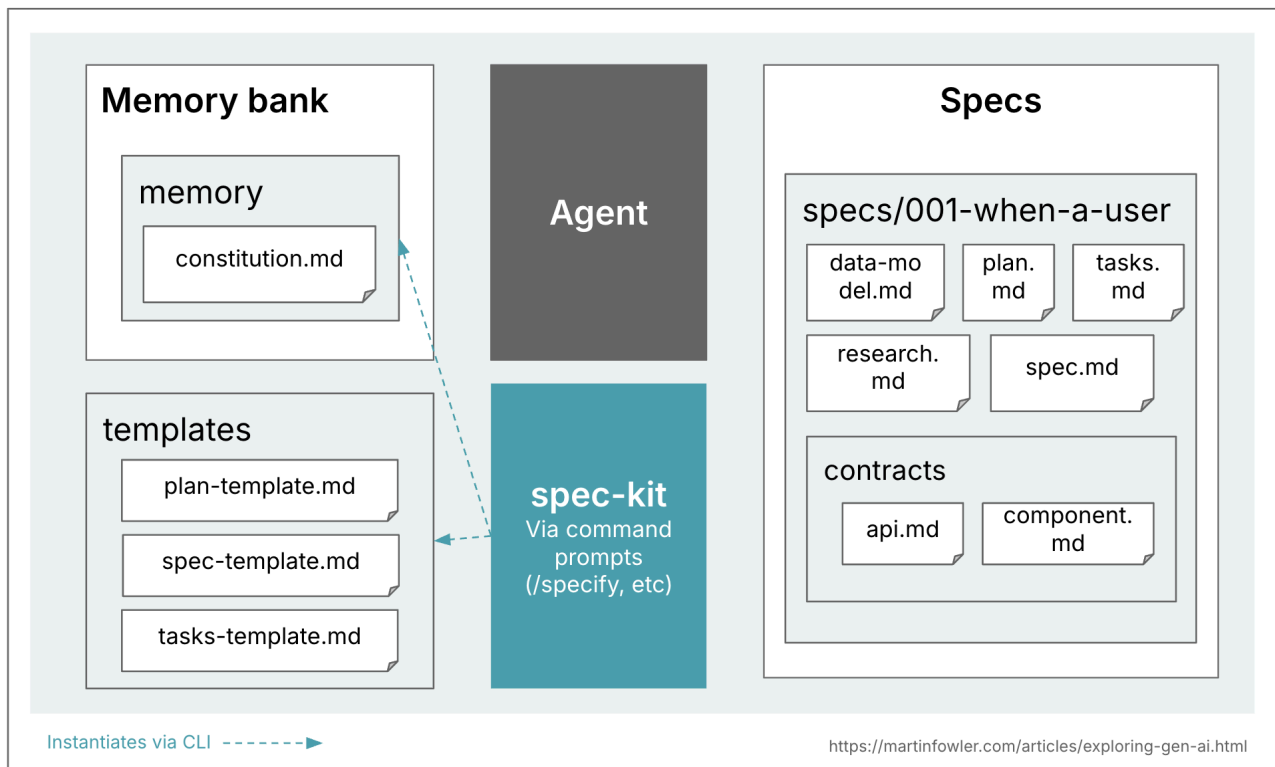
In each of the workflow steps (specify, plan, tasks), spec-kit instantiates a set of files and prompts with the help of a bash script and some templates. The workflow then makes heavy use of checklists inside of the files, to track necessary user clarifications, constitution violations, research tasks, etc. They are like a “definition of done” for each workflow step (though interpreted by AI, so there is no 100% guarantee that they will be respected).

```

spec.md × constitution.md plan.md
specs > 001-when-a-user > spec.md > # Feature Specification: Dashboard Change Impact Summary >
1      # Feature Specification: Dashboard Change Impact Summary
59     ## Requirements *(mandatory)*
72     ### Key Entities *(include if feature involves data)*
73
80
81     ## Review & Acceptance Checklist
82     *GATE: Automated checks run during main() execution*
83
84     ### Content Quality
85     - [x] No implementation details (languages, frameworks, APIs)
86     - [x] Focused on user value and business needs
87     - [x] Written for non-technical stakeholders
88     - [x] All mandatory sections completed
89
90     ### Requirement Completeness
91     - [x] No [NEEDS CLARIFICATION] markers remain
92     - [x] Requirements are testable and unambiguous
93     - [x] Success criteria are measurable
94     - [x] Scope is clearly bounded
95     - [x] Dependencies and assumptions identified
96
97     ---
98
99     ## Execution Status
100    *Updated by main() during processing*
101
102    - [x] User description parsed
103    - [x] Key concepts extracted
104    - [x] Ambiguities marked
105    - [x] User scenarios defined
106    - [x] Requirements generated
107    - [x] Entities identified
108    - [x] Review checklist passed
109
110    ---
111

```

Below is an overview to illustrate the file topology I saw in spec-kit. Note how one spec is made up of many files.

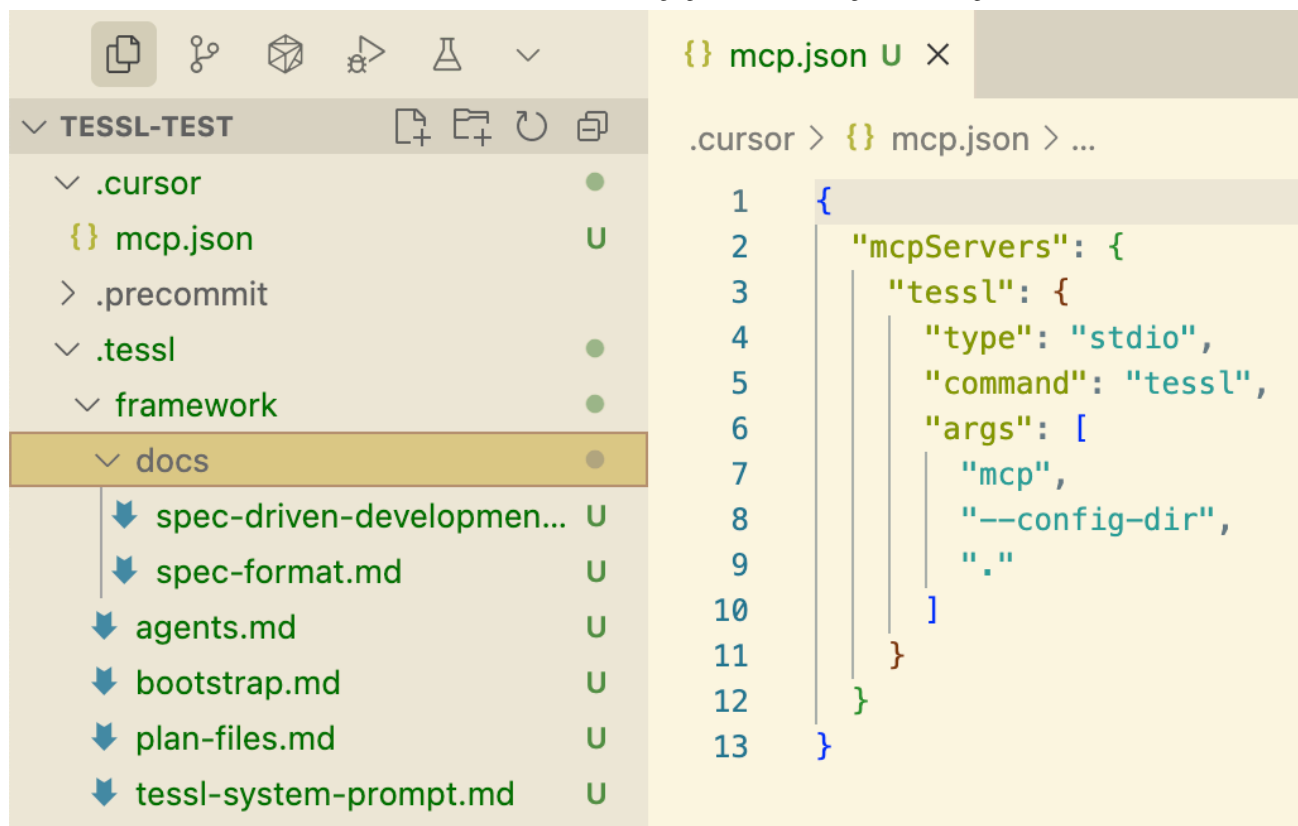


At first glance, GitHub seems to be aspiring to a spec-anchored approach (“That’s why we’re rethinking specifications — not as static documents, but as living, executable artifacts that evolve with the project. Specs become the shared source of truth. When something doesn’t make sense, you go back to the spec; when a project grows complex, you refine it; when tasks feel too large, you break them down.”) However, spec-kit creates a branch for every spec that gets created, which seems to indicate that they see a spec as a living artifact for the lifetime of a change request, not the lifetime of a feature. This community discussion is talking about this confusion. It makes me think that spec-kit is still what I would call spec-first only, not spec-anchored over time.

Tessl Framework

(Still in private beta)

Like spec-kit, the Tessl Framework is distributed as a CLI that can create all the workspace and config structure for a variety of coding assistants. The CLI command also doubles as an MCP server.



Tessl is the only one of these three tools that explicitly aspires to a spec-anchored approach, and is even exploring the spec-as-source level of SDD. A Tessl spec can serve as the main artifact that is being maintained and edited, with the code even marked with a comment at the top saying `// GENERATED FROM SPEC - DO NOT EDIT`. This is currently a 1:1 mapping between spec and code files, i.e. one spec translates into one file in the codebase. But Tessl is still in beta and they are experimenting with different versions of this, so I can imagine that this approach could also be taken on a level where one spec maps to a code component with multiple files. It remains to be seen what the alpha product will support. (The Tessl team themselves see their framework as something that is more in the future than their current public product, the Tessl Registry.)

Here is an example of a spec that I had the Tessl CLI reverse engineer (`tessl document --code ...js`) from a JavaScript file in an existing codebase:



The screenshot shows a code editor with two tabs: `_dynamic_data_renderer.spec.md` and `JS _dynamic_data_renderer.js 9, M`. The active tab is the spec file, which contains the following content:

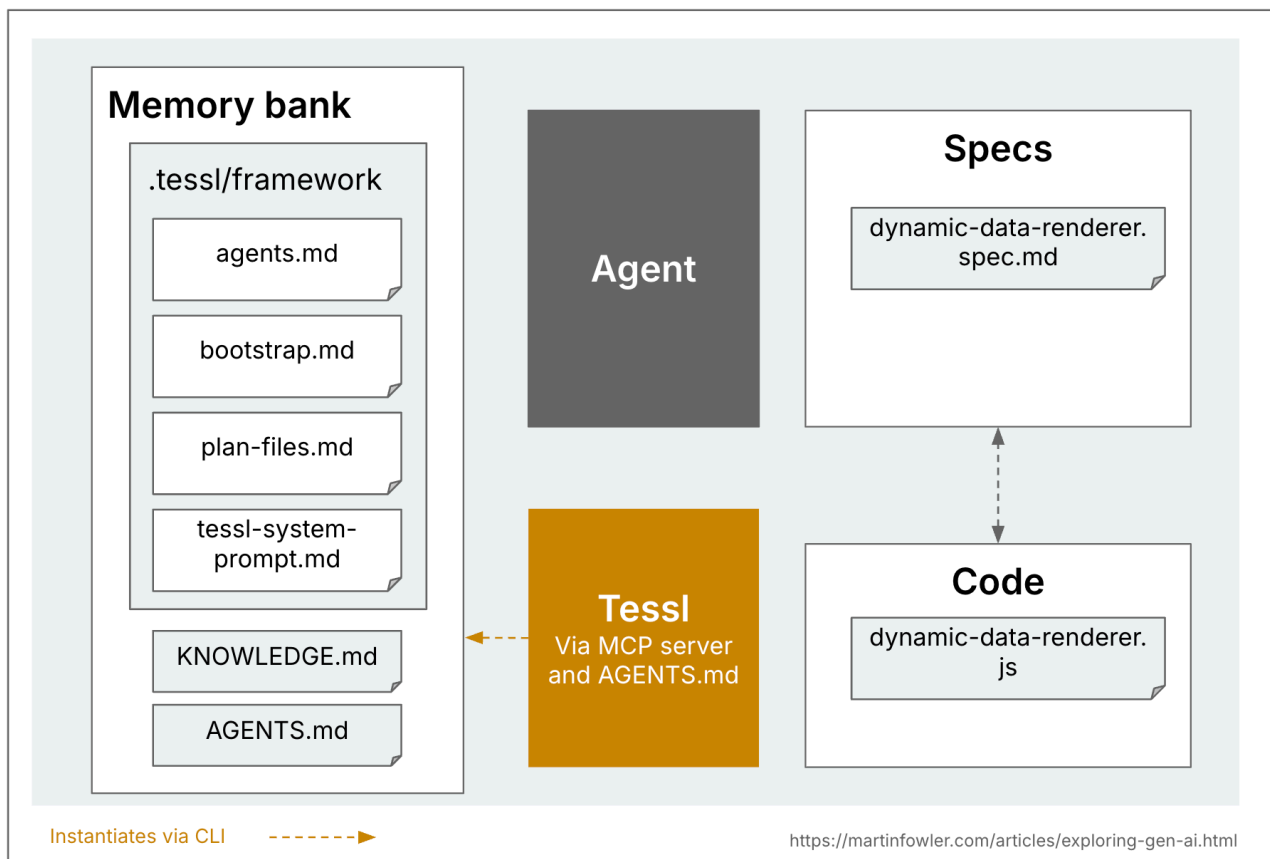
```

1  # Dynamic Data Renderer
2
3  A React component system for dynamically rendering complex data objects with intelligent type
4  detection and formatting.
5
6  ## Target
7
8  [@generate](./_dynamic_data_renderer.js)
9
10 ## Capabilities
11
12 ### Convert keys to readable text
13
14 Transforms various naming conventions into human-readable labels.
15
16 - Converts snake_case to "Snake case" [@test](./tests/_dynamic_data_renderer.test.js)
17 - Converts kebab-case to "Kebab case" [@test](./tests/_dynamic_data_renderer.test.js)
18 - Converts camelCase to "Camel case" [@test](./tests/_dynamic_data_renderer.test.js)
19 - Capitalizes first letter of converted text [@test](./tests/_dynamic_data_renderer.test.js)
20 - Trims whitespace from converted text [@test](./tests/_dynamic_data_renderer.test.js)
21
22 > ### Convert scenario data to markdown ...
23
24
25
26
27
28
29
30
31
32 > ### Render dynamic data components ...
33
34
35
36
37
38
39
40
41
42
43
44
45
46 > ### Render different value types ...
47
48
49
50
51
52
53
54
55
56
57
58
59 ## API
60
61 ``javascript { .api }
62 // Helper function to convert naming conventions to readable text
63 export const toReadableText = (key: string) => string;
64
65 // Convert scenario data to markdown format
66 export const scenarioToText = (
67   data: object,
68   exclude?: string[] | number,
69   headerLevel?: number
70 ) => string;
71
72 // Main component for dynamic data rendering
73 export const DynamicDataRenderer = ({
74   data: any,
75   exclude?: string[],
76   skipTitles?: boolean,
77   className?: string
78 }) => React.Component;
79
80 // Helper to render different value types
81 export const renderValue = (value: any, parentKey?: string) => React.Component;

```

Tags like `@generate` or `@test` seem to tell Tessel what to generate. The API section shows the idea of defining at least the interfaces that get exposed to other parts of the codebase in the spec, presumably to make sure that these more crucial parts of the generated component are fully under the control of the maintainer. Running `tessel build` for this spec generates the corresponding JavaScript code file.

Putting the specs for spec-as-source at a quite low abstraction level, per code file, probably reduces amount of steps and interpretations the LLM has to do, and therefore the chance of errors. Even at this low abstraction level I have seen the non-determinism in action though, when I generated code multiple times from the same spec. It was an interesting exercise to iterate on the spec and make it more and more specific to increase the repeatability of the code generation. That process reminded me of some of the pitfalls and challenges of writing an unambiguous and complete specification.



Observations and questions

These three tools are all labelling themselves as implementations of spec-driven development, but they are quite different from each other. So that's the first thing to keep in mind when talking about SDD, it is not just one thing.

One workflow to fit all sizes?

Kiro and spec-kit provide one opinionated workflow each, but I'm quite sure that neither of them is suitable for the majority of real life coding problems. In particular, it's not quite clear to me how they would cater to enough different problem sizes to be generally applicable.

When I asked Kiro to fix a small bug (it was the same one I used in the past to try Codex), it quickly became clear that the workflow was like using a sledgehammer to crack a nut. The requirements document turned this small bug into 4 “user stories” with a total of 16 acceptance criteria, including gems like “User story: As a developer, I want the transformation function to handle edge cases gracefully, so that the system remains robust when new category formats are introduced.”

I had a similar challenge when I used spec-kit, I wasn't quite sure what size of problem to use it for. Available tutorials are usually based on creating an application from scratch, because that's easiest for a tutorial. One of the use cases I ended up trying was a feature that would be a 3-5 point story on one of my past teams. The feature depended on a lot of code that was already there, it was supposed to build an overview modal that summarised a bunch of data from an existing dashboard. With the amount of steps spec-kit took, and the amount of markdown files it created for me to review, this again felt like overkill for the size of the problem. It was a bigger problem than the one I used with Kiro, but also a much more elaborate workflow. I never even finished the full implementation, but I think in the same time it took me to run and review the spec-kit results I could have implemented the feature with “plain” AI-assisted coding, and I would have felt much more in control.

An effective SDD tool would at the very least have to provide flexibility for a few different core workflows, for different sizes and types of changes.

Reviewing markdown over reviewing code?

As just mentioned, and as you can see in the description of the tool above, spec-kit created a LOT of markdown files for me to review. They were repetitive, both with each other, and with the code that already existed. Some contained code already. Overall they were just very verbose and tedious to review. In Kiro it was a little easier, as you only get 3 files, and it's more intuitive to understand the mental model of “requirements > design > tasks”. However, as mentioned, Kiro also was way too verbose for the small bug I was asking it to fix.

To be honest, I'd rather review code than all these markdown files. An effective SDD tool would have to provide a very good spec review experience.

False sense of control?

Even with all of these files and templates and prompts and workflows and checklists, I frequently saw the agent ultimately not follow all the instructions. Yes, the context windows are now larger, which is often mentioned as one of the enablers of spec-

driven development. But just because the windows are larger, doesn't mean that AI will properly pick up on everything that's in there.

For example: Spec-kit has a research step somewhere during planning, and it did a lot of research on the existing code and what's already there, which was great because I asked it to add a feature that built on top of existing code. But ultimately the agent ignored the notes that these were descriptions of existing classes, it just took them as a new specification and generated them all over again, creating duplicates. But I didn't only see examples of ignoring instructions, I also saw the agent go way overboard because it was too eagerly following instructions (e.g. one of the constitution articles).

The past has shown that the best way for us to stay in control of what we're building are small, iterative steps, so I'm very skeptical that lots of up-front spec design is a good idea, especially when it's overly verbose. An effective SDD tool would have to cater to an iterative approach, but small work packages almost seem counter to the idea of SDD.

How to effectively separate functional from technical spec?

It is a common idea in SDD to be intentional about the separation between functional spec and technical implementation. The underlying aspiration I guess is that ultimately, we could have AI fill in all the solutioning and details, and switch to different tech stacks with the same spec.

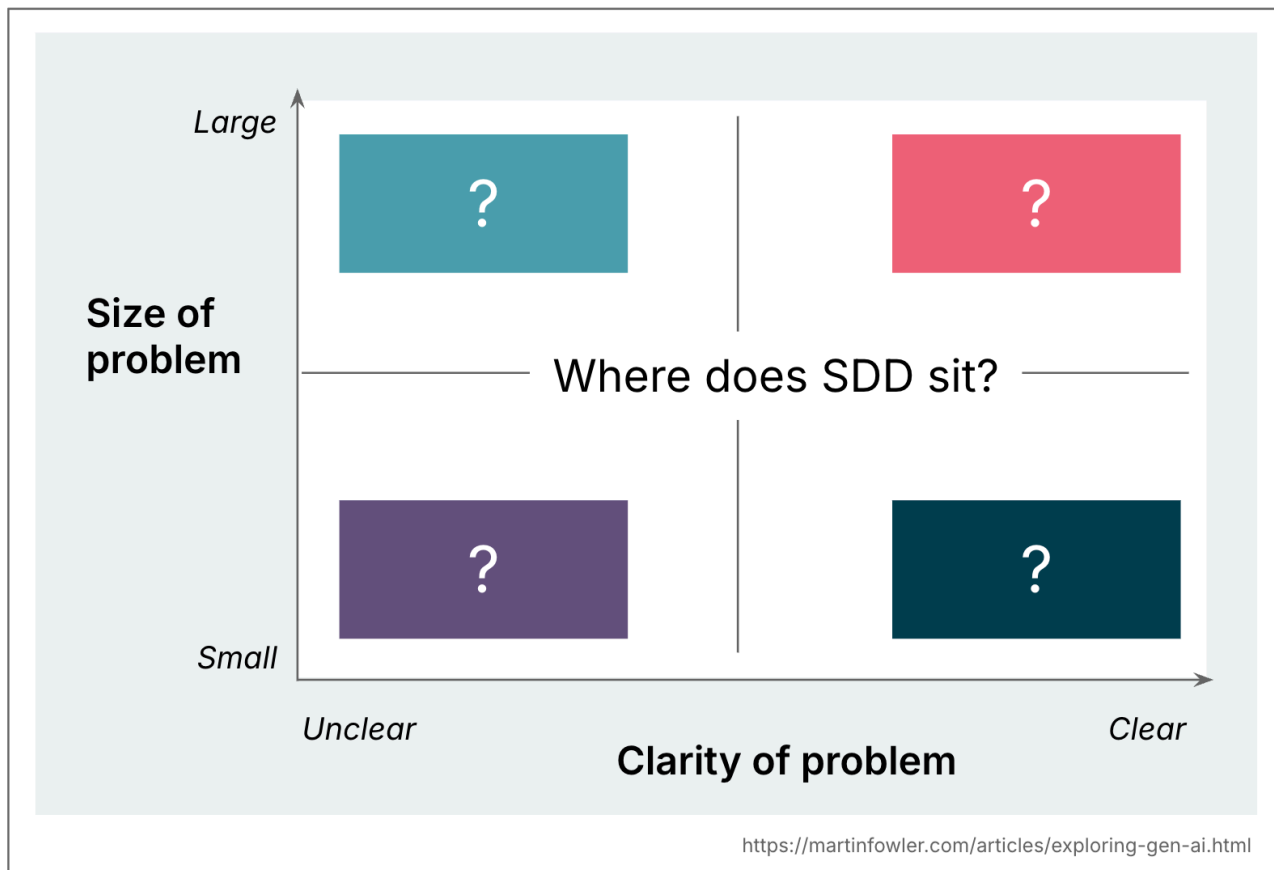
In reality, when I was trying spec-kit, I frequently got confused when to stay on the functional level, and when it was time to add technical details. The tutorial and documentation also weren't quite consistent with it, there seem to be different interpretations of what "purely functional" really means. And when I think back on the many, many user stories I've read in my career that weren't properly separating requirements from implementation, I don't think we have a good track record as a profession to do this well.

Who is the target user?

Many of the demos and tutorials for spec-driven development tools include things like defining product and feature goals, they even incorporate terms like "user story". The idea here might be to use AI as an enabler for cross-skilling, and have developers participate more heavily in requirements analysis? Or have developers pair with product people when they work on this workflow? None of this is made explicit though, it's presented as a given that a developer would do all this analysis.

In which case I would ask myself again, what problem size and type is SDD meant for? Probably not for large features that are still very unclear, as surely that would require

more specialist product and requirements skills, and lots of other steps like research and stakeholder involvement?



Spec-anchored and spec-as-source: Are we learning from the past?

While many people draw analogies between SDD and TDD or BDD, I think another important parallel to look at for spec-as-source in particular is MDD (model-driven development). I worked on a few projects at the beginning of my career that heavily used MDD, and I kept being reminded about that when I was trying out the Tessel Framework. The models in MDD were basically the specs, albeit not in natural language, but expressed in e.g. custom UML or a textual DSL. We built custom code generators to turn those specs into code.

```

Screen "Write Message" instantiates InputScreen {
  Title "XYZ";
  Intro "Here you can send a message to..."
  Section "Your Message" {
    Input "Subject" @Message.Subject
    Input "Message" @Message.Message
    Input "Create listing" ShouldListingBeCreated BOOLEAN
    ...
    Action SendMessage (
      label="Send",
      logic=@SendMessage(ShouldListingBeCreated),
      exit (if ShouldListingBeCreated then
        MessageSentAndListingCreated else MessageSent)
    )
  }
  ...
}

```

Example of a structured, parseable specification DSL from my past experience, mostly recreated from memory. Note things like references to domain model fields, inheritance from other screens for reusability of patterns, navigation logic.

Ultimately, MDD never took off for business applications, it sits at an awkward abstraction level and just creates too much overhead and constraints. But LLMs take some of the overhead and constraints of MDD away, so there is a new hope that we can now finally focus on writing specs and just generate code from them. With LLMs, we are not constrained by a predefined and parseable spec language anymore, and we don't have to build elaborate code generators. The price for that is LLMs' non-determinism of course. And the parseable structure also had upsides that we're losing now: We could provide the spec author with a lot of tool support to write valid, complete and consistent specs. I wonder if spec-as-source, and even spec-anchoring, might end up with the downsides of both MDD and LLMs: Inflexibility *and* non-determinism.

To be clear, I'm not nostalgic about my MDD experience in the past and saying "we might as well bring that back". But we should look to code-from-spec attempts in the past to learn from them when we explore spec-driven today.

Conclusions

In my personal usage of AI-assisted coding, I also often spend time on carefully crafting some form of spec first to give to the coding agent. So the general principle of spec-first is definitely valuable in many situations, and the different approaches of how to structure that spec are very sought after. They are among the top most frequently asked questions I hear at the moment from practitioners: "How do I structure my memory bank?", "How do I write a good specification and design document for AI?".

But the term “spec-driven development” isn’t very well defined yet, and it’s already semantically diffused. I’ve even recently heard people use “spec” basically as a synonym for “detailed prompt”.

Regarding the tools I’ve tried, I have listed many of my questions about their real world usefulness here. I wonder if some of them are trying to feed AI agents with our existing workflows too literally, ultimately amplifying existing challenges like review overload and hallucinations. Especially with the more elaborate approaches that create lots of files, I can’t help but think of the German compound word “Verschlimmbesserung”: Are we making something worse in the attempt of making it better?

latest article (Sep 02):
[An Accidental Blackboard](#)

previous article:
[Anchoring AI to a reference application](#)



next article:
[Assessing internal quality while coding with an agent](#)

© Martin Fowler | Disclosures

